

P.PORTO

NodeJS, ExpressJS & Bases de Dados

Programação em Ambiente Web

Índice

Introdução

Sistemas de Gestão de Bases de Dados

SGBD Relacional vs NoSQL

MongoDB

Autorização & Autenticação

Referências

Introdução

Onde guardar os dados?

- Ficheiros?
 - Json
 - XML
 - Texto
 - ..
- Bases de dados?
 - Relacionais
 - Não Relacionais
 - ..
- Serviços de terceiros?
 - Firebase
 - Cloud
 - ..

Introdução

A resposta não é simples e não há uma solução definitiva

Depende sempre do contexto/objetivos da nossa aplicação

Introdução

Exemplos:

- Utilizar ficheiros simples para armazenar imagens e vídeos é uma solução viável em programação web
- Uma base de dados relacional é uma boa escolha para sistemas de transações bancárias
- Um base de dados NoSQL pode ser adequada para um blog

Introdução

As aplicações Web necessitam de lidar com dados introduzidos pelos utilizadores

As operações mais comuns sobre os dados denominam-se CRUD

- **Create** – operação de criação de dados
- **Read** – operação de leitura de dados
- **Update** – operação de edição de dados
- **Delete** – operação de eliminação de dados

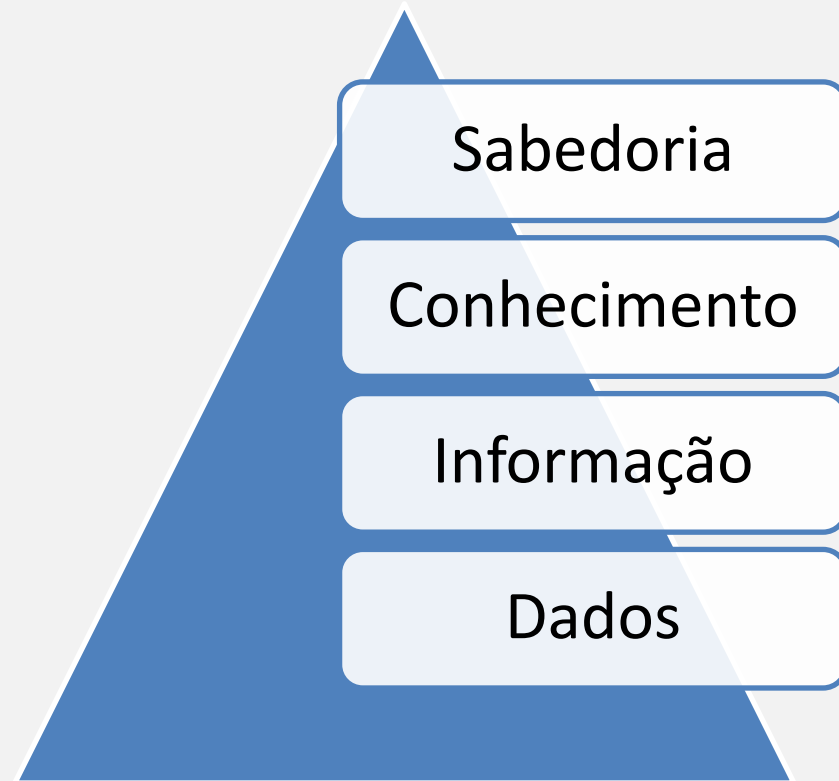
Introdução

Podemos usar operações CRUD para criar relatórios/views para utilizadores e/ou imaterializar processos de negócio em bases de dados

Podemos usar operações sobre os dados guardados de forma a acrescentar valor aos dados

Introdução

Apesar de comuns as operações CRUD não são por si as funções mais interessantes sobre dados em aplicações web



Introdução

Que tipo de restrições temos sobre os dados?

- Consistência
- Características das transações
- Disponibilidade
- Replicação
- Tolerância a falhas

As respostas às questões acima podem modificar as recomendações do slide anterior

Introdução

ACID é um conceito que se refere a quatro propriedades de uma transação de dados um sistema de base de dados relacional:

- **A**tomicidade
- **C**onsistência
- **I**solamento
- **D**urabilidade

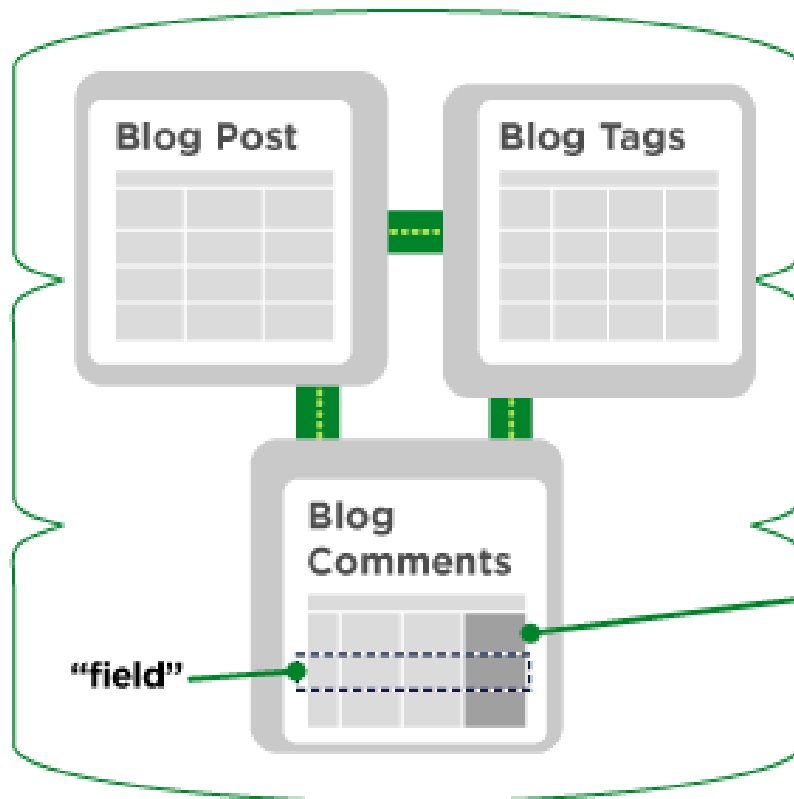
Bases de dados NoSQL podem não cumprir

Sistemas de Gestão de Bases de Dados

Sistema de gestão de base de dados relacionais
(SGBD Relacionais)

Um SGBD relacional baseia-se num conjunto de estruturas de dados com relações entre si definidos num schema

Sistemas de Gestão de Bases de Dados



A relational database table organizes structured data fields into defined columns.

Sistemas de Gestão de Bases de Dados

- As bases de dados relacionais implicam o uso de SQL
- Tem uma estrutura rígida e bem definida para os dados
- Esta abordagem tem algumas vantagens alguns tipo de aplicações:
 - Sistemas baseados em transações
 - Alta consistência de dados

Sistemas de Gestão de Bases de Dados

SQL significa Structured Query Language

- permite criar, aceder e manipular bases de dados
- tornou-se um padrão do American National Standards Institute (ANSI) em 1986, e da International Organization for Standardization (ISO) em 1987

O SQL é uma linguagem para consultar e gerir bases de dados. Parece simples, mas pode tornar-se bastante complexa

Sistemas de Gestão de Bases de Dados

Exemplos de comandos SQL

- `SELECT * FROM table_name WHERE condition;`
ex: `SELECT * FROM Customers WHERE Country='Portugal';`
- `INSERT INTO table_name (column1, column2, column3, ...) VALUES (value1, value2, value3, ...);`
–
ex: `INSERT INTO Customers (CustomerName, Address, City, Country) VALUES ('John', 'ESTG', 'Felgueiras', 'Portugal');`
- `DELETE FROM table_name WHERE condition;`
ex: `DELETE FROM Customers WHERE CustomerName='John';`
- `INSERT INTO table2 (SELECT * FROM table1) WHERE condition;`
- `SELECT column_name(s) FROM table1 FULL OUTER JOIN table2 ON table1.column_name = table2.column_name WHERE condition;`

Sistemas de Gestão de Bases de Dados

Exemplos de SGBD?

- Oracle
- SQLServer
- MySQL
- MariaDB
- Postgres
- ...

Sistemas de Gestão de Bases de Dados

NoSQL, com o significado "no SQL," são uma alternativa aos SGDB relacionas tradicionais

São geralmente usadas para trabalhar com grandes quantidades de dados distribuídos

Não possuem um esquema de dados rígido embora seja possível atribuir um esquema aos dados

Sistemas de Gestão de Bases de Dados

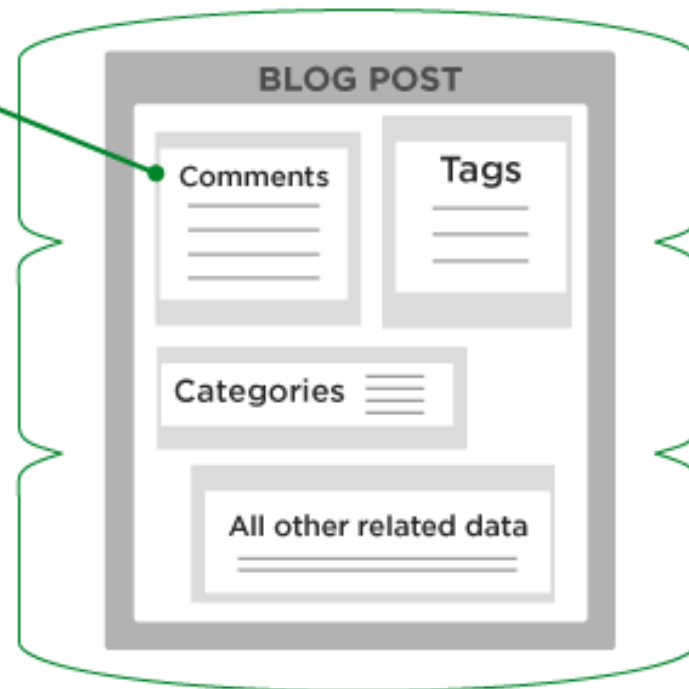
As bases de dados NoSQL caracterizam-se por serem eventualmente consistentes ao contrário de base de dados relacionais que são sempre consistentes

A terminologia eventualmente consistente tem implicações no tratamento de transições concorrentes e na consistência dos dados

Sistemas de Gestão de Bases de Dados

NoSQL – Linguagem No SQL

A non-relational database does not incorporate the table model. Instead, data can be stored in a single document file.



Sistemas de Gestão de Bases de Dados

Exemplo

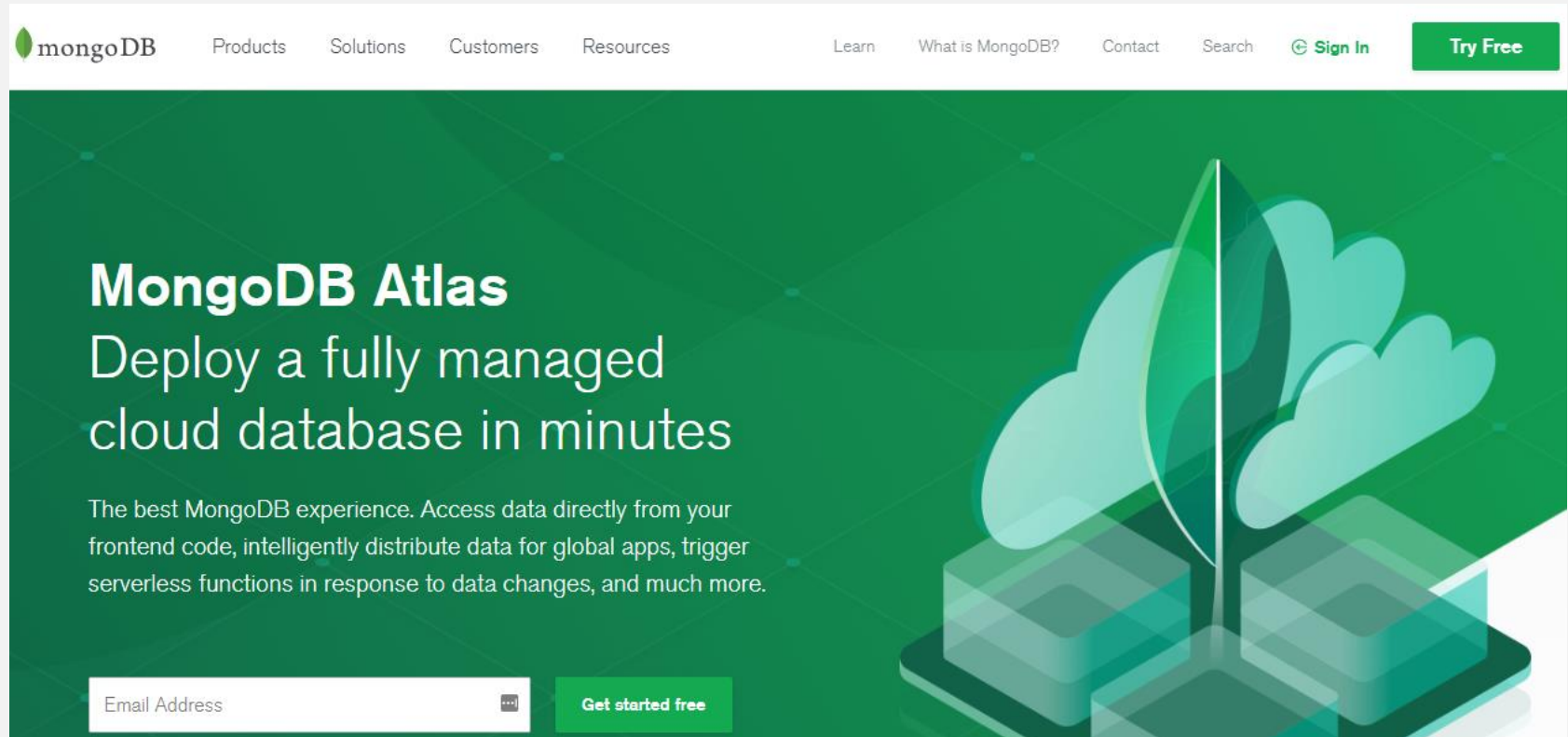
- MongoDB
- CouchDB
- Cassandra
- Firebase
- Riak
- ...

SGBD Relacionais / NoSQL

NoSQL

- Prós:
 - NoSQL não depende de schemas
 - Modelo de dados sem schema é mais rico que pares de chave / valor
 - consistência eventual
 - Melhor adaptados para escalabilidade dos dados
 - fornecem bom desempenho
- Contras:
 - Normalmente, não há transações ou associações ACID

MongoDB

The image shows the MongoDB Atlas landing page. The header is white with the MongoDB logo on the left and navigation links (Products, Solutions, Customers, Resources, Learn, What is MongoDB?, Contact, Search, Sign In) in the center. A green 'Try Free' button is on the right. The main content area has a green background with a subtle grid pattern. On the left, the text 'MongoDB Atlas' is in large white font, followed by 'Deploy a fully managed cloud database in minutes'. Below this is a paragraph: 'The best MongoDB experience. Access data directly from your frontend code, intelligently distribute data for global apps, trigger serverless functions in response to data changes, and much more.' At the bottom left is an 'Email Address' input field with a small icon to its right. To the right of the input field is a green 'Get started free' button. On the right side of the main content area is a 3D illustration of a stylized tree with green leaves and a white trunk, standing on a base of stacked, semi-transparent cubes.

Fonte: <https://www.mongodb.com/>

MongoDB

É um exemplo de base de dados NoSQL

É parte integrante da MEAN (Mongo Express Angular Node) Stack para desenvolvimento web

Tal como os componentes da MEAN stack é desenvolvido em JavaScript



Fonte: <https://i0.wp.com/blog.adeel.io/wp-content/uploads/2016/11/mean-stack-1.png?fit=640%2C296>

MongoDB

É adequado no caso de:

- Necessitar apenas de persistir e ler dados
- O formato dos objectos pode mudar de objeto para objeto
- Quero distribuir os dados por vários servidores de dados
- Existem muitos objetos com propriedades diferentes

MongoDB

Terminologia

- Coleção – equivalente a tabelas em sql
- Documentos – equivalentes a linhas em tabelas sql
- Fields – equivalentes a campos em tabelas sql
- Schema – embora o mongoDB não use schemas, podemos forçar o uso de schemas via software aplicacional
- Modelos – Construtores de alto nível que usam um schema virtual para gerar objetos a guardar numa coleção

MongoDB

(NoSQL) Document oriented Database

- MongoDB guarda os dados num formato flexível, estilo ficheiros JSON
- Objectos da mesma coleção podem conter nos seus campos um número de propriedades diferente

O driver mongoose

- O modelo de documento é mapeado para os objetos no código-fonte da aplicação, facilitando o trabalho com os dados
- Consultas ad hoc, indexação e agregação em tempo real fornecem maneiras poderosas de aceder e analisar seus dados

MongoDB

Instale módulo para comunicar com base de dados MongoDB:

- `npm install mongoose --save`

Importar o módulo em expressJS

- `var mongoose = require('mongoose')`

MongoDB

Conexão à base de dados

```
var mongoose = require('mongoose');  
  
mongoose.Promise = global.Promise;  
  
mongoose.connect('mongodb://localhost/produtos')  
  .then(() => console.log('connection succesful'))  
  .catch((err) => console.error(err));
```

MongoDB

Model Schema

```
var mongoose = require('mongoose');

var EmployeeSchema = new mongoose.Schema({
  name: String,
  address: String,
  position: String,
  salary: Number,
  updated_at: { type: Date, default: Date.now },
});

module.exports = mongoose.model('Employee', EmployeeSchema);
```

MongoDB

Encontrar um objeto

```
var mongoose = require("mongoose");
var Employee = require("../models/Employee");

var employeeController = {};

// Show employee by id
employeeController.show = function(req, res) {
  Employee.findOne({_id: req.params.id}).exec(function (err, employee) {
    if (err) {
      console.log("Error:", err);
    }
    else {
      res.render("../views/employees/show", {employee: employee});
    }
  });
};
```

MongoDB

Guardar Objetos

```
var mongoose = require("mongoose");
var Employee = require("../models/Employee");

var employeeController = {};

// Save new employee
employeeController.save = function(req, res) {
  var employee = new Employee(req.body);

  employee.save(function(err) {
    if(err) {
      console.log(err);
      res.render("../views/employees/create");
    } else {
      console.log("Successfully created an employee.");
      res.redirect("/employees/show/"+employee._id);
    }
  });
};
```

MongoDB

Formulário para editar objeto

```
var mongoose = require("mongoose");
var Employee = require("../models/Employee");

var employeeController = {};

// Edit an employee
employeeController.edit = function(req, res) {
  Employee.findOne({_id: req.params.id}).exec(function (err, employee) {
    if (err) {
      console.log("Error:", err);
    }
    else {
      res.render("../views/employees/edit", {employee: employee});
    }
  });
};
```


MongoDB

Update de objetos

```
var mongoose = require("mongoose");
var Employee = require("../models/Employee");

var employeeController = {};

// Update an employee
employeeController.update = function(req, res) {
  Employee.findByIdAndUpdate(req.params.id, { $set: { name: req.body.name, address: req.body.address,
    position: req.body.position, salary: req.body.salary }}, { new: true }, function (err, employee) {
    if (err) {
      console.log(err);
      res.render("../views/employees/edit", {employee: req.body});
    }
    res.redirect("/employees/show/"+employee._id);
  });
};
```

MongoDB

Apagar de objetos

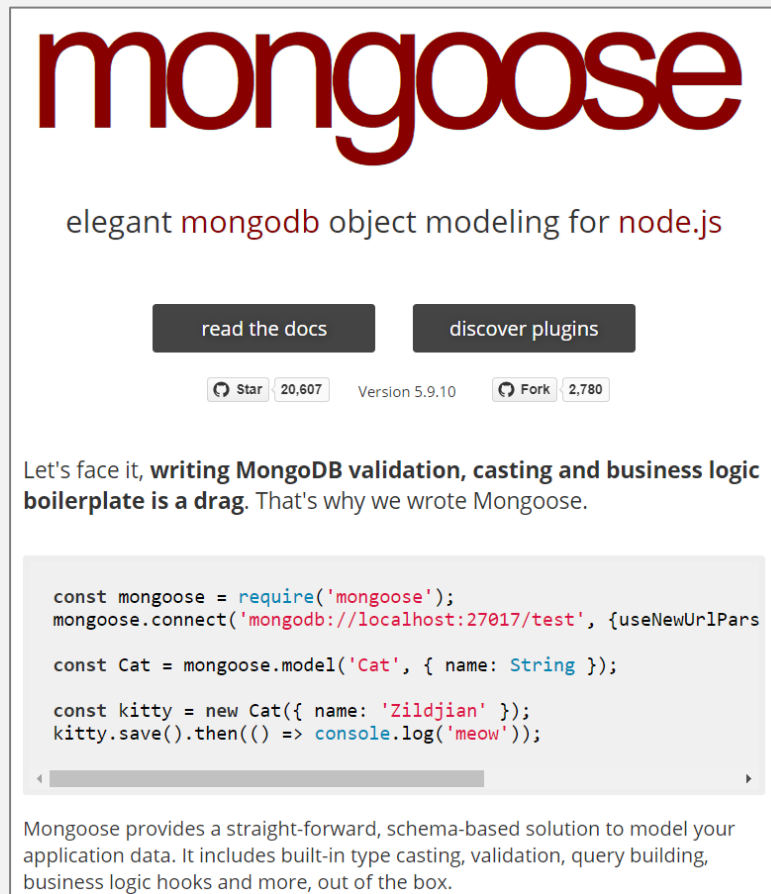
```
var mongoose = require("mongoose");
var Employee = require("../models/Employee");

var employeeController = {};

// Delete an employee
employeeController.delete = function(req, res) {
  Employee.remove({_id: req.params.id}, function(err) {
    if(err) {
      console.log(err);
    }
    else {
      console.log("Employee deleted!");
      res.redirect("/employees");
    }
  });
};
```

MongoDB

Documentação Mongoose



The screenshot shows the Mongoose.js website. At the top is the word "mongoose" in a large, red, lowercase font. Below it is the tagline "elegant mongodb object modeling for node.js". There are two dark buttons: "read the docs" and "discover plugins". Below these are statistics: "Star 20,607", "Version 5.9.10", and "Fork 2,780". A paragraph of text reads: "Let's face it, **writing MongoDB validation, casting and business logic boilerplate is a drag.** That's why we wrote Mongoose." Below this is a code block with a light gray background and a scrollbar, containing JavaScript code for connecting to MongoDB and creating a model. At the bottom, a paragraph states: "Mongoose provides a straight-forward, schema-based solution to model your application data. It includes built-in type casting, validation, query building, business logic hooks and more, out of the box."

mongoose

elegant **mongodb** object modeling for **node.js**

[read the docs](#) [discover plugins](#)

[Star](#) 20,607 [Version](#) 5.9.10 [Fork](#) 2,780

Let's face it, **writing MongoDB validation, casting and business logic boilerplate is a drag.** That's why we wrote Mongoose.

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/test', {useNewUrlParser: true});

const Cat = mongoose.model('Cat', { name: String });

const kitty = new Cat({ name: 'Zildjian' });
kitty.save().then(() => console.log('meow'));
```

Mongoose provides a straight-forward, schema-based solution to model your application data. It includes built-in type casting, validation, query building, business logic hooks and more, out of the box.

Fonte: <https://mongoosejs.com/>

Que tipo de base de dados deve usada?

A melhor ferramenta para solucionar o problema que nós for proposto de maneira eficiente respondendo a todas as restrições do nosso problema

Não existe uma resposta única para esse problema!

Padrão MVC

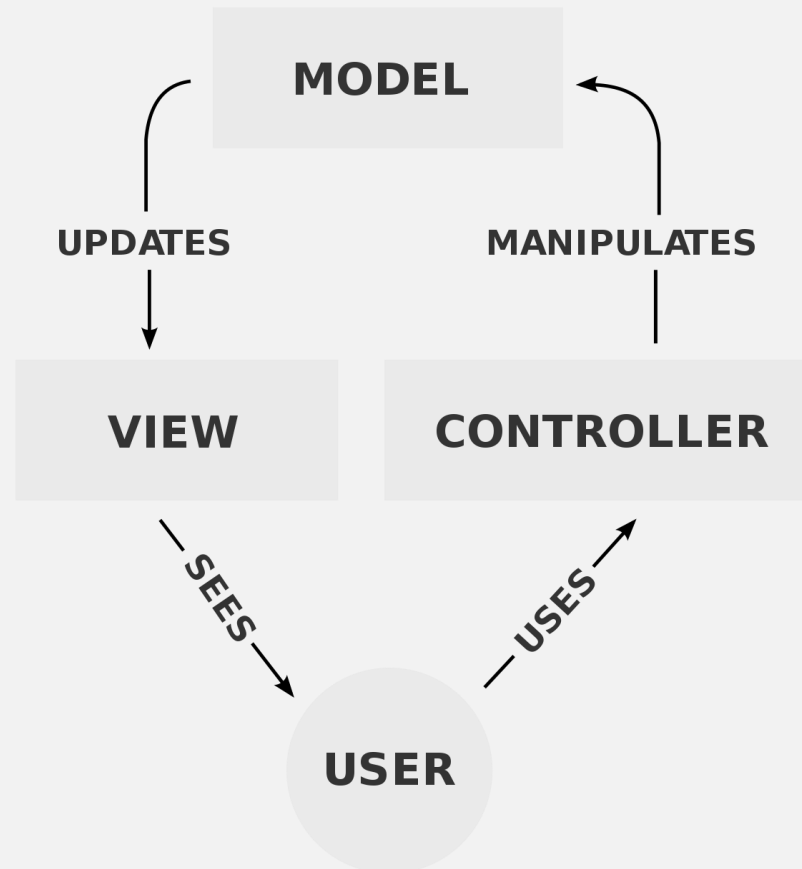
Com conhecimento sobre *template engines* e sistemas de base de dados as aplicações web aumentam a sua complexidade

Como devemos organizar as aplicações?

- Usar padrões de software
 - **Model** – Modelos dos dados
 - **View** – Apresentação dos dados
 - **Controllers** – Controladores que ligam dados do modelo às views

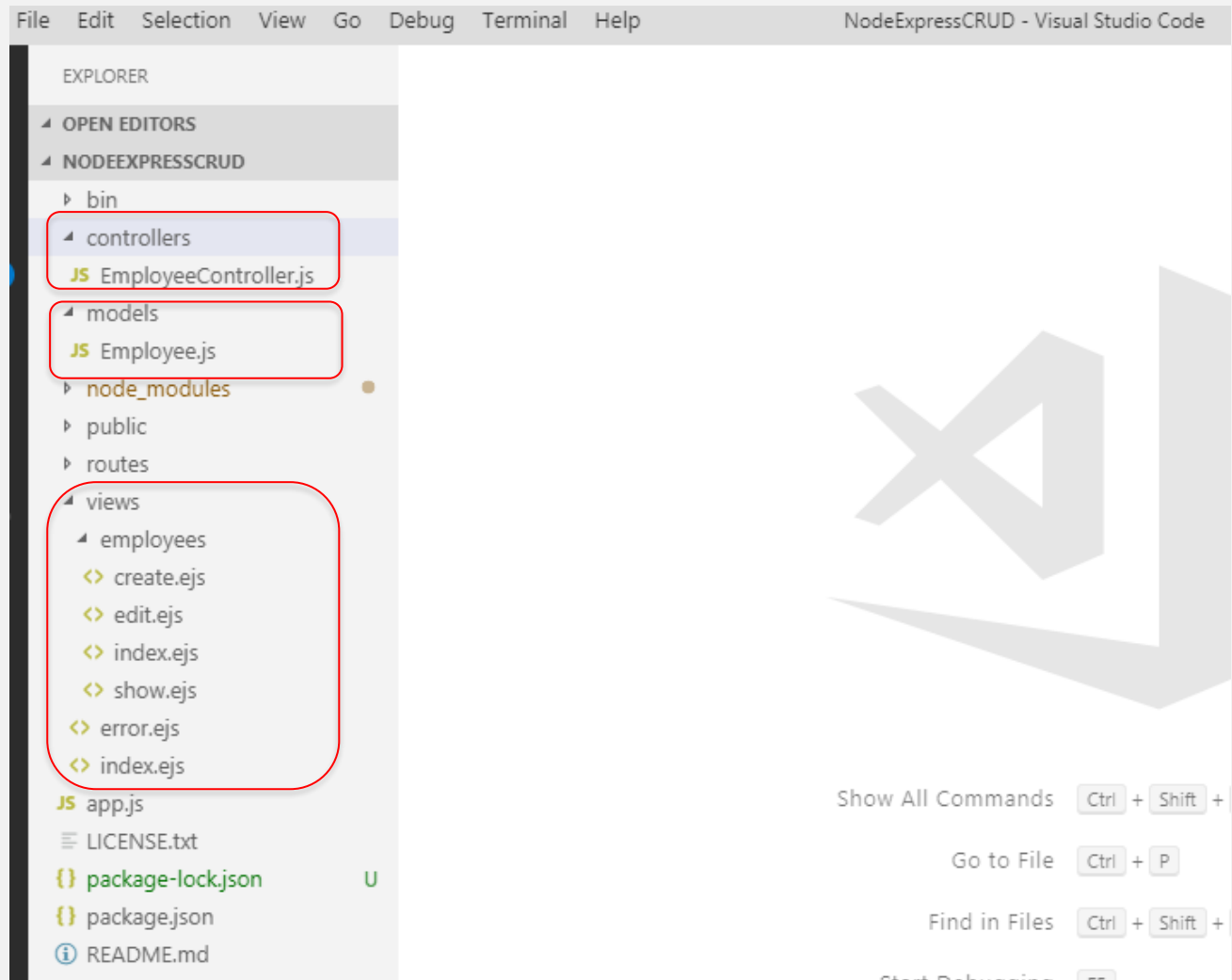
Padrão MVC

MVC



Padrão MVC

MVC



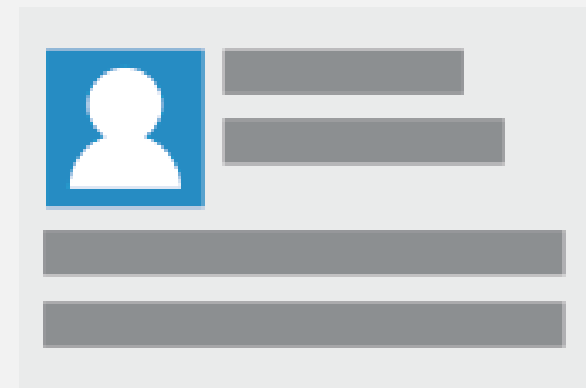
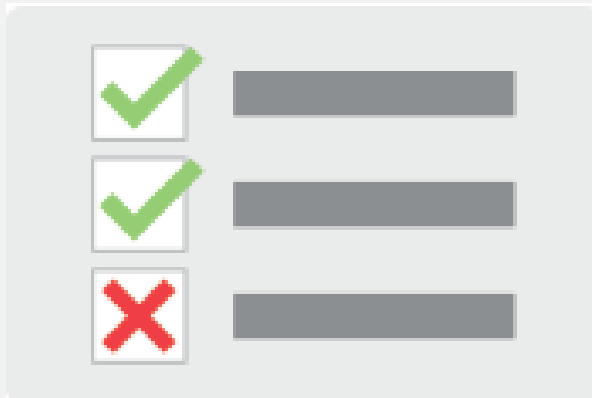
Autenticação & Autorização

O que é Autenticação?

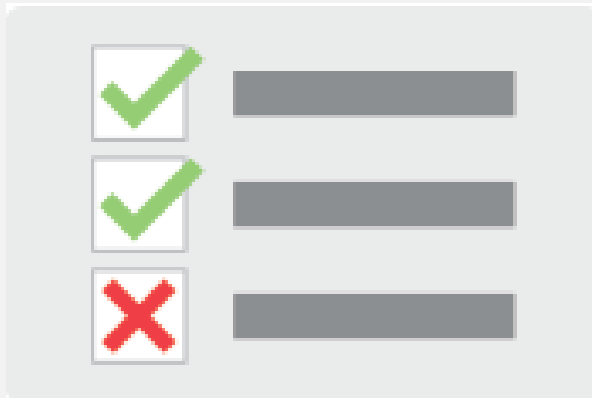
O que é Autorização?

São diferentes?

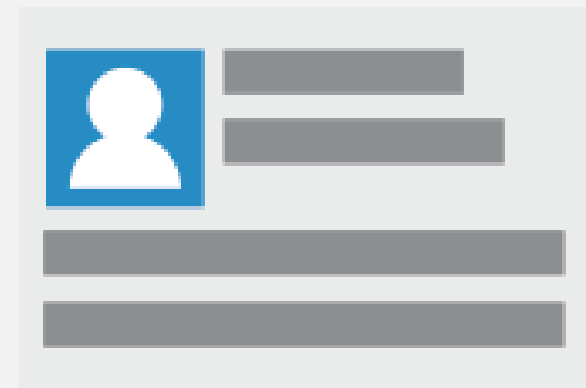
Node.js Autenticação & Autorização



Node.js Autenticação & Autorização



Autorização



Autenticação

Node.js Autenticação & Autorização

Autenticação e autorização são dois conceitos diferentes em aplicações web

Autenticação diz respeito a identificação do utilizador, deve assegurar-nos que o utilizador é quem diz ser

Autorização diz respeito às permissões de um utilizador na aplicação (ex: utilizadores admin e guest)

Node.js Autenticação & Autorização

Podemos na nossa aplicação em ExpressJS definir uma autenticação básica usando middleware:

- Enviar para o servidor informação de login em http headers
- Verificar no servidor as informações nos headers e fornecer acesso aos recursos

O transporte da informação http deve no mínimo ser feito através do protocolo HTTPS

Node.js Autenticação & Autorização

Para autenticação devemos usar o módulo passport em nodeJS

- <http://www.passportjs.org/>

Para autorização devemos usar user roles definir o acesso de cada tipo de utilizador. Existe já middleware integrado no express como é o caso do módulo ACL

O primeiro deverá efetuar o login dos nossos utilizadores enquanto o segundo restringirá o acesso a recursos de acordo com as permissões estabelecidas

Referências

Express

- <https://expressjs.com/>
- <https://ejs.co/>
- https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs

MongoDB

- <https://mongoosejs.com/docs/index.html>

MySQL

- <https://www.npmjs.com/package/mysql>

PassPort

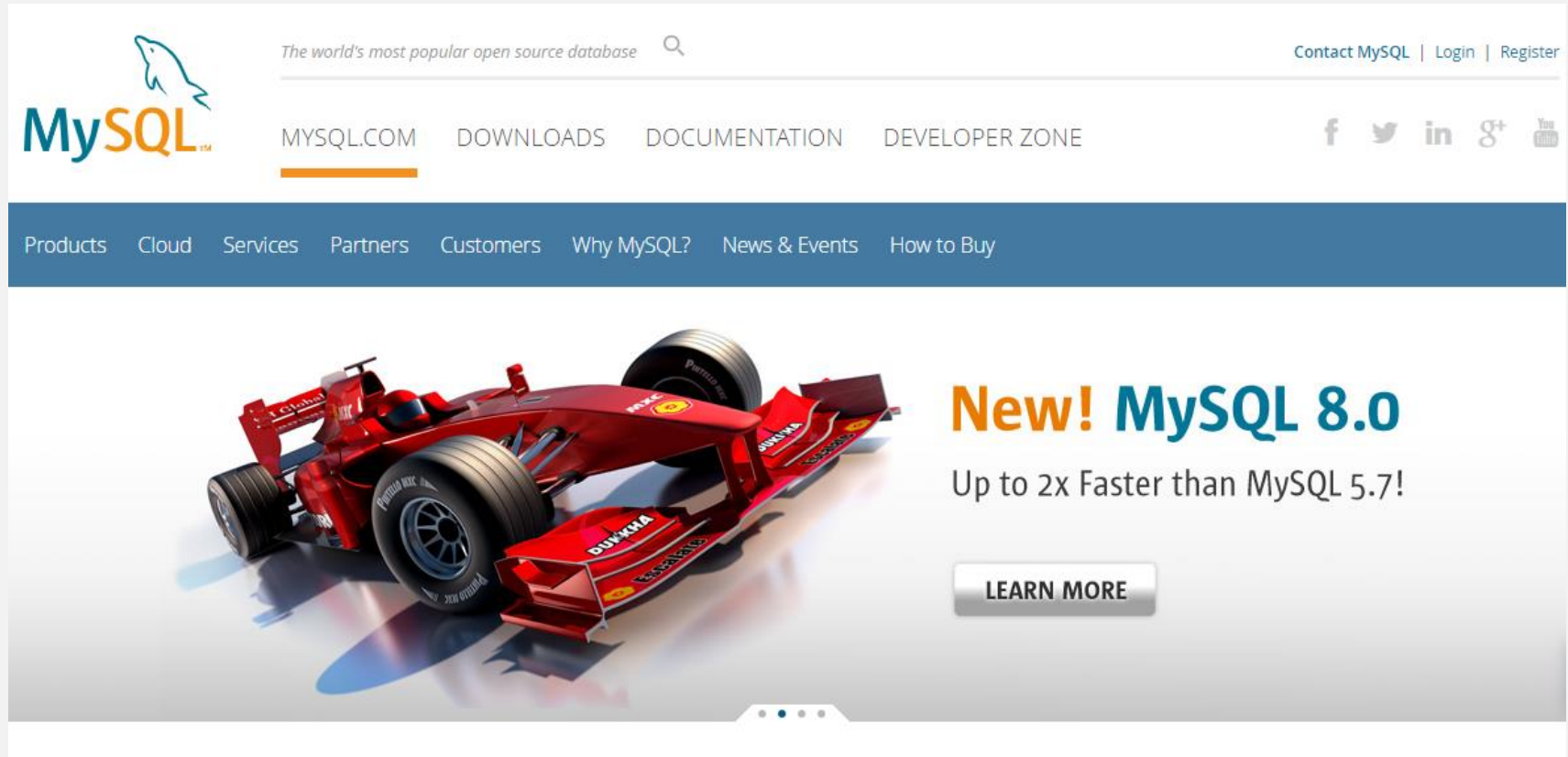
- <http://www.passportjs.org/>

P.PORTO

NodeJS, ExpressJS & Bases de Dados

Programação em Ambiente Web

MySQL



The screenshot shows the MySQL website homepage. At the top left is the MySQL logo, which includes a blue fish icon and the text "MySQL™". To its right is the tagline "The world's most popular open source database" and a search icon. On the far right of the top navigation bar are links for "Contact MySQL", "Login", and "Register". Below the tagline is a horizontal menu with "MYSQL.COM" (underlined), "DOWNLOADS", "DOCUMENTATION", and "DEVELOPER ZONE". To the right of this menu are social media icons for Facebook, Twitter, LinkedIn, Google+, and YouTube. A dark blue horizontal bar contains a secondary menu with links: "Products", "Cloud", "Services", "Partners", "Customers", "Why MySQL?", "News & Events", and "How to Buy". The main content area features a large image of a red Ferrari Formula 1 car on the left. To the right of the car, the text "New! MySQL 8.0" is displayed in large, bold letters, with "New!" in orange and "MySQL 8.0" in blue. Below this, it says "Up to 2x Faster than MySQL 5.7!". At the bottom right of this section is a grey button with the text "LEARN MORE".

MySQL™

The world's most popular open source database

MYSQL.COM DOWNLOADS DOCUMENTATION DEVELOPER ZONE

Products Cloud Services Partners Customers Why MySQL? News & Events How to Buy

New! MySQL 8.0

Up to 2x Faster than MySQL 5.7!

LEARN MORE

Fonte: <https://www.mysql.com/>

MySQL

Podemos usar uma base de dados MySQL com node.js e express

Instale módulo para comunicar com base de dados MySQL:

- `npm install mysql --save`

Módulo a importar

- `var mysql = require('mysql')`

MySQL

- Conexão à base de dados

```
var mysql = require('mysql');

var con = mysql.createConnection({
  host: "localhost",
  user: "yourusername",
  password: "yourpassword"
});

con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
});
```

MySQL

Inserção na base de dados

```
con.connect(function(err) {  
  if (err) throw err;  
  console.log("Connected!");  
  var sql = "INSERT INTO UC (uc, course) VALUES ('PAW', 'LEI/LSIRC')";  
  con.query(sql, function (err, result) {  
    if (err) throw err;  
    console.log("1 record inserted");  
  });  
});
```

MySQL

Update na base de dados

```
con.connect(function(err) {  
  if (err) throw err;  
  var sql = "UPDATE UC SET course = 'LEI/LSRIC' WHERE course = 'ESTG'";  
  con.query(sql, function (err, result) {  
    if (err) throw err;  
    console.log(result.affectedRows + " record(s) updated");  
  });  
});
```

MySQL

Queries à base de dados

```
con.connect(function(err) {  
  if (err) throw err;  
  //Select all customers and return the result object:  
  con.query("SELECT * FROM UC", function (err, result, fields) {  
    if (err) throw err;  
    console.log(result);  
  });  
});
```

MySQL

Remoções à base de dados

```
con.connect(function(err) {  
  if (err) throw err;  
  var sql = "DELETE FROM UC WHERE uc = 'PAW'";  
  con.query(sql, function (err, result) {  
    if (err) throw err;  
    console.log("Number of records deleted: " + result.affectedRows);  
  });  
});
```

MySQL

ORM significa Object Relational Mapping:

- Podemos usar ORM para automatizar a criação, acesso, introdução, edição e remoção de dados
- ORMs abstraem e automatizam muitas tarefas no uso de base de dados relacionais

MySQL

ORM:

- Criam a partir da definição dos objetos as relações na base de dados
- Têm estruturas nos seus módulos para as operações mais comuns com os dados
- São geralmente simples de utilizar
- Têm algumas desvantagens:
 - Performance em queries mais complexas que impliquem relações entre objetos
 - A api pode tornar-se complexa para operações específicas sobre os dados que não estão directamente mapeadas na API do ORM

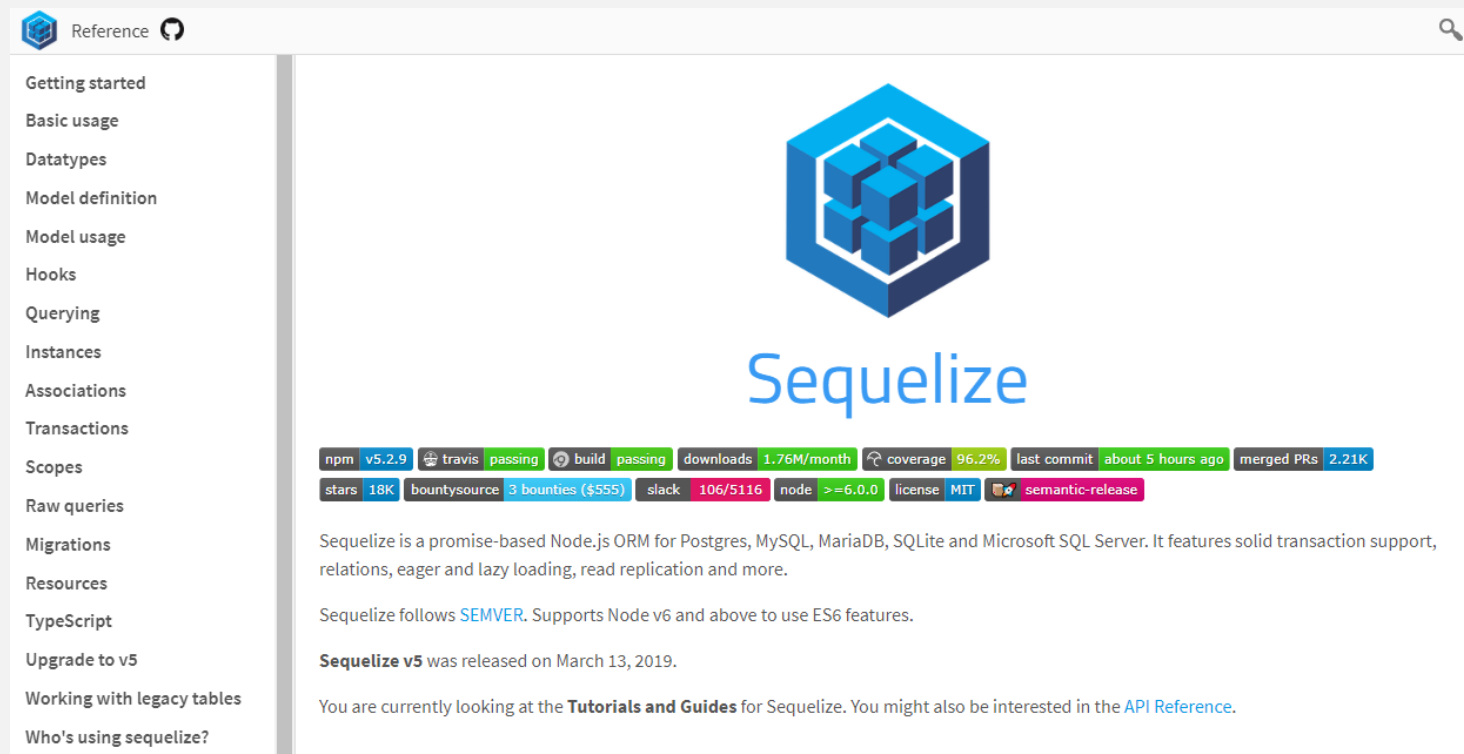
MySQL

Exemplo de ORM em node.js:

- Sequelize - <http://docs.sequelizejs.com/>
- Node-orm2 - <https://github.com/dresende/node-orm2>
- Bookshelf - <https://bookshelfjs.org/>
- Objection.js - <https://github.com/Vincit/objection.js>

MySQL

Sequelize ORM



Reference

Getting started
Basic usage
Datatypes
Model definition
Model usage
Hooks
Querying
Instances
Associations
Transactions
Scopes
Raw queries
Migrations
Resources
TypeScript
Upgrade to v5
Working with legacy tables
Who's using sequelize?


Sequelize

npm v5.2.9 travis passing build passing downloads 1.76M/month coverage 96.2% last commit about 5 hours ago merged PRs 2.21K
stars 18K bountysource 3 bounties (\$555) slack 106/5116 node >=6.0.0 license MIT semantic-release

Sequelize is a promise-based Node.js ORM for Postgres, MySQL, MariaDB, SQLite and Microsoft SQL Server. It features solid transaction support, relations, eager and lazy loading, read replication and more.

Sequelize follows [SEMVER](#). Supports Node v6 and above to use ES6 features.

Sequelize v5 was released on March 13, 2019.

You are currently looking at the **Tutorials and Guides** for Sequelize. You might also be interested in the [API Reference](#).

Fonte: <http://docs.sequelizejs.com/>